LEARN BEING FORWARD

Programming — C# Programming

Exception handling & delegates

COMPLETE LEARNING GUIDE

What You'll Learn

Exception handling is C#'s structured mechanism for responding to runtime errors without letting them crash an entire application, and delegates are its type-safe way of treating functions as values — passing behavior around as data, the same conceptual foundation that underlies event handling, LINQ's lambda-based operators, and asynchronous callback patterns throughout .NET. The two topics pair naturally in this sub-module because robust error handling and callback-driven code are both about structuring what happens when control flow doesn't follow the straightforward, linear path — an exception interrupts normal flow to jump to a handler, and a delegate lets code decide at runtime which specific function should run in response to an event. This sub-module covers try/catch/finally mechanics, the exception class hierarchy and custom exceptions, practical exception-handling discipline, delegate fundamentals, multicast delegates and events, and the built-in Func/Action/Predicate delegate types.

1. try/catch/finally Mechanics

Code that might throw goes inside a try block; one or more catch blocks (each specifying an exception type) handle specific kinds of failure, checked in order from top to bottom, with the first matching type's catch block running; an optional finally block runs unconditionally afterward, whether or not an exception occurred, and is the standard place for cleanup code (closing a file, releasing a lock) that must happen regardless of success or failure. Multiple catch blocks for the same try let you handle different exception types differently — more specific exception types should be caught before more general ones, since C# matches the FIRST catch block whose type matches (or is a base type of) the thrown exception.

```csharp
try {
    int[] numbers = { 1, 2, 3 };
    Console.WriteLine(numbers[5]); // throws IndexOutOfRangeException
}
catch (IndexOutOfRangeException ex) {
    Console.WriteLine($"Index error: {ex.Message}");
}
catch (Exception ex) { // more general — must come AFTER the specific catch above
    Console.WriteLine($"Unexpected error: {ex.Message}");
}
finally {
    Console.WriteLine("Cleanup runs regardless of success or failure");
}
```

2. Exception Hierarchy and Custom Exceptions

Every exception in .NET derives, directly or indirectly, from System.Exception, with common built-in subtypes including ArgumentException (and its more specific ArgumentNullException), InvalidOperationException, and IndexOutOfRangeException, each representing a distinct category of failure. Defining a custom exception (deriving from Exception, or a more specific built-in exception when appropriate) lets application code signal domain-specific failure conditions clearly — an InsufficientFundsException communicates intent far better than a generic Exception or InvalidOperationException would, both to a human reading a stack trace and to calling code that wants to catch that specific failure mode without accidentally also catching unrelated errors.

```csharp
public class InsufficientFundsException : Exception {
    public InsufficientFundsException(string message) : base(message) {}
}

public class Account {
    public decimal Balance { get; private set; } = 100;

    public void Withdraw(decimal amount) {
        if (amount > Balance) {
            throw new InsufficientFundsException($"Cannot withdraw {amount}; balance is {Balance}");
        }
        Balance -= amount;
    }
}

try {
    new Account().Withdraw(500);
} catch (InsufficientFundsException ex) {
    Console.WriteLine(ex.Message);
}
```

3. Best Practices: Specific Catches and Preserving Stack Traces

Catching the broadest possible type (catch (Exception ex)) and silently swallowing it (an empty catch block, or one that only logs and continues as if nothing happened) hides real bugs and makes production issues far harder to diagnose — catch only the specific exception types you can genuinely recover from, and let everything else propagate upward where it can be properly logged or surfaced. When an exception needs to be re-thrown after some handling, bare throw; (with no exception variable) preserves the original stack trace, while throw ex; resets it to the re-throw point, destroying the information about where the error ACTUALLY originated — this is a subtle but important distinction most developers get wrong the first time they encounter it.

- An empty or logging-only catch block that swallows an exception without re-throwing or genuinely handling it is almost always a bug waiting to surface somewhere else, later, with no useful trace.
- throw; (bare, inside a catch block) preserves the original stack trace; throw ex; resets it — always prefer bare throw; when re-throwing.

4. Delegates: Defining and Invoking

A delegate is a type-safe reference to a method matching a specific signature (return type and parameter list) — declaring delegate int Operation(int a, int b); defines a new delegate TYPE that can hold a reference to ANY method matching int Method(int, int), and a variable of that delegate type can then be assigned a specific method and invoked exactly like calling that method directly. This is C#'s type-safe equivalent of C's function pointers, but with the compiler fully verifying signature compatibility at compile time rather than allowing arbitrary pointer-level mismatches.

```csharp
delegate int Operation(int a, int b); // defines a delegate TYPE

class MathOps {
    static int Add(int a, int b) => a + b;
    static int Multiply(int a, int b) => a * b;
}

Operation op = MathOps.Add; // delegate variable holding a reference to Add
Console.WriteLine(op(3, 4)); // 7

op = MathOps.Multiply; // reassign to a different matching method
Console.WriteLine(op(3, 4)); // 12
```

5. Multicast Delegates and Events

A delegate variable can hold references to MULTIPLE methods simultaneously (a multicast delegate) by combining them with the += operator — invoking the delegate then calls every attached method in the order they were added. This is exactly the mechanism behind C#'s event keyword, which wraps a multicast delegate with restricted access (only the declaring class can invoke/raise the event; subscribers can only += or -= their own handler, not invoke it directly or clear others' handlers) — the standard pattern for publish/subscribe notifications throughout .NET, including every UI framework's click/change event handlers.

```csharp
public class Button {
    public event Action Clicked; // event based on the built-in Action delegate

    public void SimulateClick() => Clicked?.Invoke(); // ?. guards against zero subscribers
}

var button = new Button();
button.Clicked += () => Console.WriteLine("Handler 1 responding");
button.Clicked += () => Console.WriteLine("Handler 2 responding");

button.SimulateClick();
// Handler 1 responding
// Handler 2 responding
```

6. Lambda Expressions as Delegates

Any delegate variable can be assigned a lambda expression (a concise, inline, unnamed function) instead of a reference to an existing named method — (a, b) => a + b is a lambda matching any delegate type expecting two parameters and returning their sum's type, and this is overwhelmingly the most common way delegates are actually used in modern C# code, especially as arguments to LINQ methods (Where(x => x > 0)) and event handlers for short, one-off logic that doesn't justify a separate named method.

7. Func<>, Action<>, and Predicate<>: Built-In Delegate Types

Rather than declaring a custom delegate type for every signature you need, .NET provides three generic built-in delegate families covering the overwhelming majority of real use cases: Func<T1, ..., TResult> represents a method taking the listed parameter types and returning TResult (the LAST type parameter is always the return type); Action<T1, ...> represents a method with no return value (void); and Predicate<T> represents a method taking a T and returning bool, specifically used for filtering/testing logic (though Func<T, bool> is functionally equivalent and often used interchangeably in modern code).

```csharp
Func<int, int, int> add = (a, b) => a + b;        // takes two ints, returns an int
Action<string> log = message => Console.WriteLine($"LOG: {message}");
Predicate<int> isEven = n => n % 2 == 0;

Console.WriteLine(add(3, 4));   // 7
log("Something happened");       // LOG: Something happened
Console.WriteLine(isEven(10));  // True
```

Common Pitfalls

- Catching the broadest Exception type and swallowing it silently instead of catching specific, recoverable exception types and letting the rest propagate.
- Using throw ex; to re-throw inside a catch block, which resets the stack trace and destroys information about where the error actually originated — use bare throw; instead.
- Ordering catch blocks from most general to most specific, causing the general catch to always match first and the specific one to become unreachable.
- Forgetting the null-conditional operator (?.) when invoking an event with no subscribers, risking a NullReferenceException.
- Confusing Action (no return value) with Func (has a return value) and picking the wrong one for a given delegate parameter.
- Never unsubscribing an event handler (-=) when the subscribing object's lifetime is shorter than the publisher's, causing a memory leak by keeping the subscriber alive indefinitely.

Key Takeaways

- catch blocks are checked top to bottom; order them from most specific to most general exception type.
- Bare throw; inside a catch block preserves the original stack trace; throw ex; resets it — always prefer bare throw; when re-throwing.
- Delegates are type-safe references to methods matching a specific signature; events are a restricted, publish/subscribe-safe wrapper around multicast delegates.
- Lambda expressions are the dominant modern way to supply delegate values inline, especially for LINQ and short event handlers.
- Func<>, Action<>, and Predicate<> cover almost every common delegate signature without needing a custom delegate type declaration.

Learn More

Microsoft Learn — Exceptions and exception handling
https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/exceptions/

Microsoft Learn — Delegates
https://learn.microsoft.com/en-us/dotnet/csharp/delegates-overview